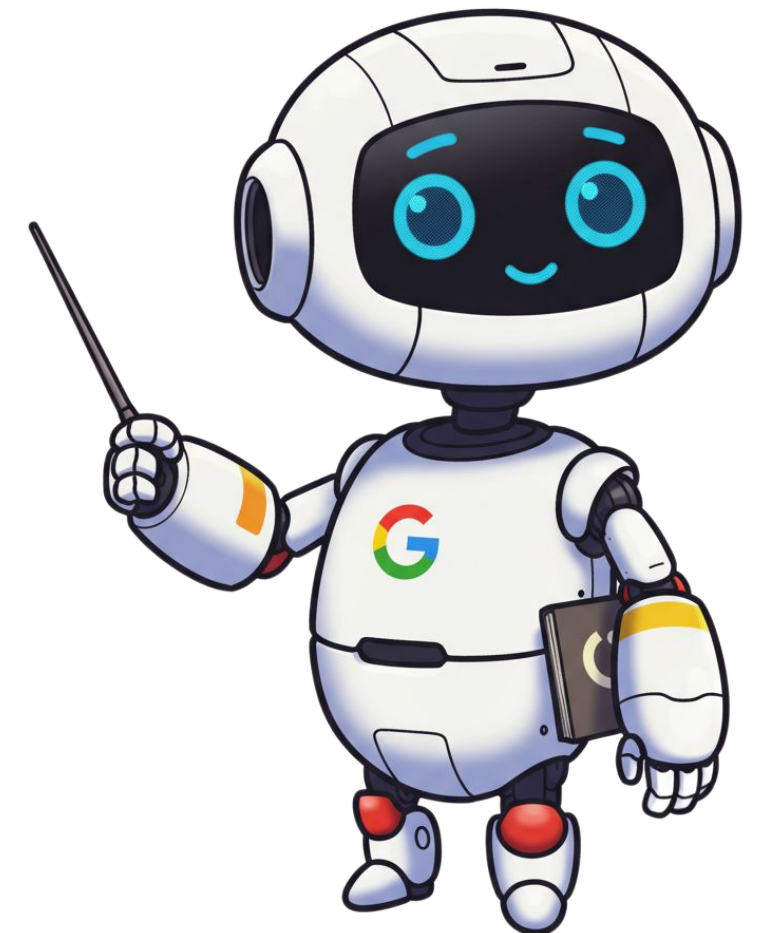


Plan before you build

Deterministic Planning Patterns for AI Agents

Dan Dobrin

Google



In the age of agentic AI, we tend to dive straight into implementation. Connect an LLM, add some tools and resources, then let the system ‘figure it out’

Not very... “architectural”





App Architect
Google Cloud

X @ddobrin

in dan-dobrin

The Elephant in the Room

Agentic non-determinism — the unspoken challenge every LLM orchestration system faces

Unpredictable Paths

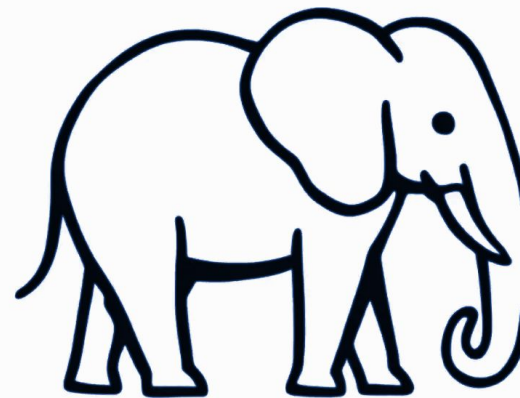
Same input, different execution path every run

Untestable Behavior

Traditional unit tests simply don't apply to agents

Hidden Cost Multiplier

Token usage w/out guardrails compounds fast



Debugging Black Box

No stack trace for “why did the agent decide this?”

Infinite Loop Risk

Agents chasing their own tail with no exit condition

Model variability

Same prompt generates widely different answers

Core tension: Agents must be autonomous enough to be useful, yet constrained enough to be reliable

Observability Shift

From deterministic software metrics to non-deterministic agent reasoning analysis

Software Observability

**Deterministic
w/Code as Source of Truth**

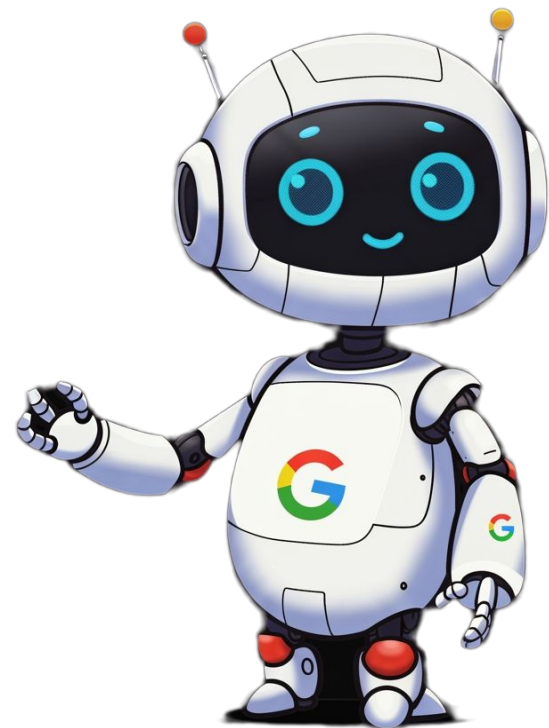
- CPU Utilization
- Memory Limits
- Network Latency



Agent Observability

**Non-Deterministic
w/Reasoning as Source of Truth**

- Tool Call Sequence
- Reasoning Path
- Decision Quality
- Time-to-first-token
- Token Consumption



AI Agents

01

(Non)Determinism In Agents

02

Agentic Planning Patterns

03

Introducing the LLM Council Pattern

“No single model is always right — a council of models is more reliable than any one genius.”

(paraphrasing Andrej Karpathy/George Box)



The Problem

Single LLMs hallucinate with confidence — no built-in checks or balance



The Council

Multiple specialized agents deliberate, challenge, and synthesize answers



The Outcome

Higher accuracy, explainable reasoning, and reduced single-point of failure



Why LLM Council?

How do you know which model is best for your query?

The Problem

No single model wins across all tasks

**Without structured evaluation,
you're guessing**

The Architecture Insight

Separate generation from evaluation

**Clear optionality across
generator and reviewer roles**

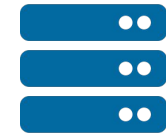
Architecture

LLM Council — Multi-Model Deliberation Platform



Multi-Model Consensus

Single question to multiple LLMs simultaneously, orchestrates anonymous peer review, models rank and analyze each other's responses



5-Stage Deliberation Engine

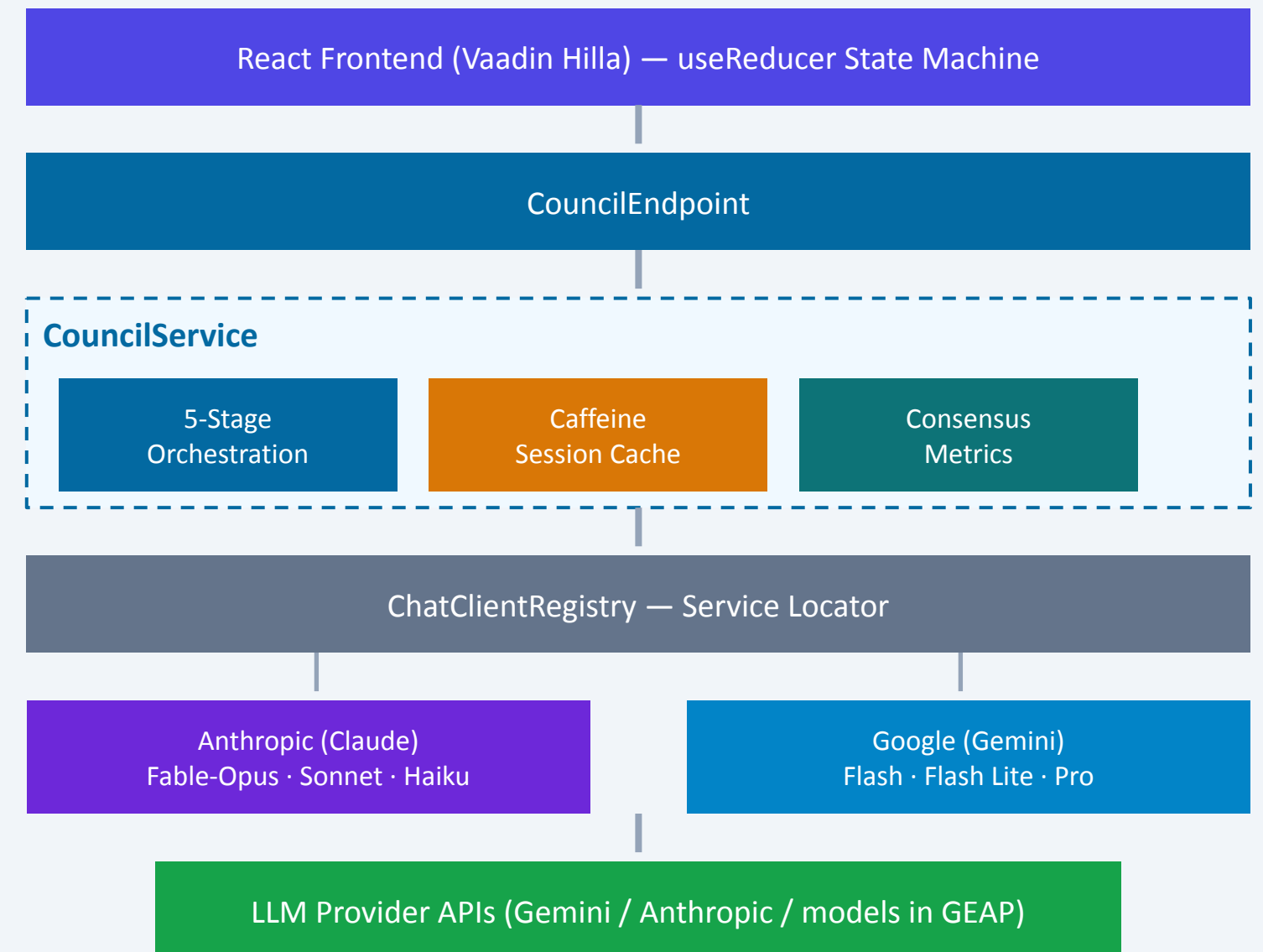
CouncilService drives individual stages. Reactive Flux/Mono chains throughout.



Provider-Agnostic Registry

Auto-collects all ChatClientProvider beans. Supports models via direct API or Vertex AI — configurable. Uses mathematical consensus metrics

Layered Architecture





Design Principles

Four foundational concepts that shape how the LLM Council achieves reliable consensus

Anonymity

Models evaluate content on merit alone

No model knows which response is its own

Structured Deliberation

Fixed multi-stage pipeline ensures every perspective is heard and evaluated before reaching a conclusion

Collective Intelligence

Agreement points surface high-confidence insights

Disagreement points highlight areas of genuine uncertainty

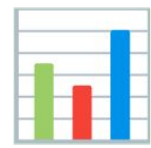
Provider Independence

All LLM providers are interchangeable members

The architecture is model-agnostic by design

Key use-cases

Where does it help me?



Investment & financial research



Complex multi-factor decisions



Fact verification & cross-check



Creative brainstorming at scale



LLM model evaluation and testing

Pros & Cons

Multi-model deliberation — consensus through structured anonymous peer review

ADVANTAGES

- ✓ Multi-Model Consensus
- ✓ Anonymous Evaluation
- ✓ 3 Flexible Flow Modes
- ✓ Provider-Agnostic Design
- ✓ Full Observability Stack

TRADE-OFFS

- ✗ Multiplied API Cost & Latency
- ✗ Parser Fragility
- ✗ Volatile Session State
- ✗ Chairman Single Point of Decision
- ✗ Overall performance to be improved

Next frontier: planning

Determinism vs Non-Determinism

Two opposing paradigms for agent planning in LLM orchestration systems

DETERMINISTIC

Execution order fixed at compile time

- ✓ **Compile-Time DAG**
- ✓ **Zero Planning LLM Calls**
- ✓ **Predictable & Reproducible**
- ✓ **Preset Compositions**

Patterns: ADK Workflow

NON-DETERMINISTIC

Execution order decided at runtime by an LLM planner

- ✗ **Runtime LLM Planning**
- ✗ **3-5 Planning Calls / Run**
- ✗ **Natural-Language Control**
- ✗ **Adaptive Partial Workflows**

Patterns: ADK Supervisor LLM



The next frontier: planning

From strict compliance all the way to bounded flexibility

◀ Strict Compliance

Bounded Flexibility ▶

Hybrid approach

Workflows

Deterministic Directed
Acyclic Graphs (DAGs)

0 planning LLM calls

SupervisorLLM

LLM-orchestrated
delegation to sub-agents

1 call per step



The next frontier: planning

From strict compliance all the way to bounded flexibility

◀ Strict Compliance

Hybrid approach

Bounded Flexibility ▶

Workflows

Deterministic Directed
Acyclic Graphs (DAGs)

0 planning LLM calls

HTN

Hierarchical Task Network
planning

1 call · classify only

SupervisorLLM

LLM-orchestrated
delegation to sub-agents

1 call per step



The next frontier: planning

From strict compliance all the way to bounded flexibility

◀ Strict Compliance

Hybrid approach

Bounded Flexibility ▶

Workflows

Deterministic Directed
Acyclic Graphs (DAGs)

0 planning LLM calls

HTN

Hierarchical Task Network
planning

1 call · classify only

Utility AI

Scored decision-making
across weighted options

0 calls · scoring fn

SupervisorLLM

LLM-orchestrated
delegation to sub-agents

1 call per step



The next frontier: planning

From strict compliance all the way to bounded flexibility

◀ Strict Compliance

Hybrid approach

Bounded Flexibility ▶

Workflows

Deterministic Directed
Acyclic Graphs (DAGs)

0 planning LLM calls

HTN

Hierarchical Task Network
planning

1 call · classify only

GOAP

Goal-Oriented Action
Planning via A* graph search

0 calls · keyword rules

Utility AI

Scored decision-making
across weighted options

0 calls · scoring fn

SupervisorLLM

LLM-orchestrated
delegation to sub-agents

1 call per step



The next frontier: planning

From strict compliance all the way to bounded flexibility

◀ Strict Compliance

Hybrid approach

Bounded Flexibility ▶

Workflows

Deterministic Directed
Acyclic Graphs (DAGs)

0 planning LLM calls

HTN

Hierarchical Task Network
planning

1 call · classify only

GOAP

Goal-Oriented Action
Planning via A* graph search

0 calls · keyword rules

Utility AI

Scored decision-making
across weighted options

0 calls · scoring fn

LLM Compiler

LLM-authored task DAG,
deterministically verified

1 call · authors the DAG

SupervisorLLM

LLM-orchestrated
delegation to sub-agents

1 call per step



The next frontier: planning

From strict compliance all the way to bounded flexibility

◀ Strict Compliance

Hybrid approach

Bounded Flexibility ▶

Workflows

Deterministic Directed
Acyclic Graphs (DAGs)

0 planning LLM calls

HTN

Hierarchical Task Network
planning

1 call · classify only

GOAP

Goal-Oriented Action
Planning via A* graph search

0 calls · keyword rules

Utility AI

Scored decision-making
across weighted options

0 calls · scoring fn

LLM Compiler

LLM-authored task DAG,
deterministically verified

1 call · authors the DAG

SupervisorLLM

LLM-orchestrated
delegation to sub-agents

1 call per step

falls back to GOAP · validation fails

Workflow Pattern



Architecture

ADK Workflow — Deterministic Fixed-Mode Orchestration



Compile-Time Agent DAGs

Execution order hard-coded using Google ADK's SequentialAgent and ParallelAgent. Fully predictable, zero planning calls.



3 Preset Compositions

Full (8 stages, max parallelism), Quick (4 stages, skip analysis), and Rank (3 stages, minimal).



5 Reusable Agents

5 agents (response, ranking, agreement, disagreement, synthesis) + 3 pure-computation aggregation actions

Full Deliberation DAG

SequentialAgent

Stage 1: Initial Response (all members, LLM)

ParallelAgent — Analysis Phase

Peer
Ranking

Agreement
Analysis

Disagreement
Analysis

ParallelAgent — Aggregation Phase

Final
Synthesis

Agg.
Rankings

Agg.
Agreements

Agg.
Disagreements

Stream Complete → Flux<StageEvent>

Zero planning LLM calls — DAG fixed at bean creation time

Pros & Cons

Fixed DAG workflow vs. dynamic LLM-supervised orchestration

ADVANTAGES

Zero Planning Overhead

No supervisor LLM calls — the DAG is fixed at bean creation time, eliminating 3–5 extra API calls per deliberation

Fully Deterministic

Execution path is 100% predictable and reproducible — same input always produces the same agent traversal order

Lower Cost

Only the actual council agent LLM calls are billed — no planning token costs, reducing per-deliberation expense

Compile-Time Validation

DAG structure validated when Spring beans are wired — misconfigurations fail at startup, not at runtime

Shared Agent Codebase

Identical 8 agent implementations used by both Workflow and Supervisor modes — zero duplication between strategies

TRADE-OFFS

Fixed to 3 Presets Only

Only full, quick, and rank modes available — adding a new composition requires code changes and redeployment

No Natural-Language Control

Cannot accept arbitrary instructions like "just find agreements" — user must pick from the predefined modes

No Partial Workflows

Each preset runs its entire DAG — cannot skip or add individual agents dynamically based on user intent

Rigid Parallelism

Parallel/sequential grouping is compile-time fixed — cannot adapt concurrency based on runtime conditions or data

No Span Auto-Cleanup

Unlike Supervisor mode, the Workflow mode does not auto-close the deliberation span in doFinally — requires explicit save call

Deterministic compile-time orchestration: predictable execution with zero planning cost

SupervisorLLM Pattern



Supervisor Algorithm

LLM-driven iterative planning — core loop structure and design invariants

ALGORITHM LOOP

1 OBSERVE

Build full context: current state keys + agent catalog + user instruction



2 SUPERVISOR LLM

Select agent(s) for this step, or emit DONE when complete

DONE ✓



3 EXECUTE AGENTS

Dispatch selected agents concurrently; collect structured outputs into state



4 UPDATE STATE

Write agent output keys into shared state map; return to step 1



DESIGN INVARIANTS

Stateless Planning

Each iteration re-reads the complete context from scratch. No memory between calls — the prompt IS the state

Probabilistic Dispatch

Agent selection is non-deterministic. Same instruction may produce different execution paths — no fixed DAG.

Emergent Parallelism

LLM declares multiple agents per step (ex AGENTS: synthesis, rankings). Concurrency emerges from decisions — not a pre-wired DAG.

Bounded Termination

DONE requires 3-layer safety validation: workflow rules, chain rules, scope filter. Hard iteration cap prevents runaway loops.

Architecture

Supervisor LLM Planning Pattern — LLM Council Orchestration



Supervisor LLM

An LLM that flexible and dynamically decides which agents to run at each step. Receives agent catalog, completed outputs, and user instruction.



3-Layer Safety Nets

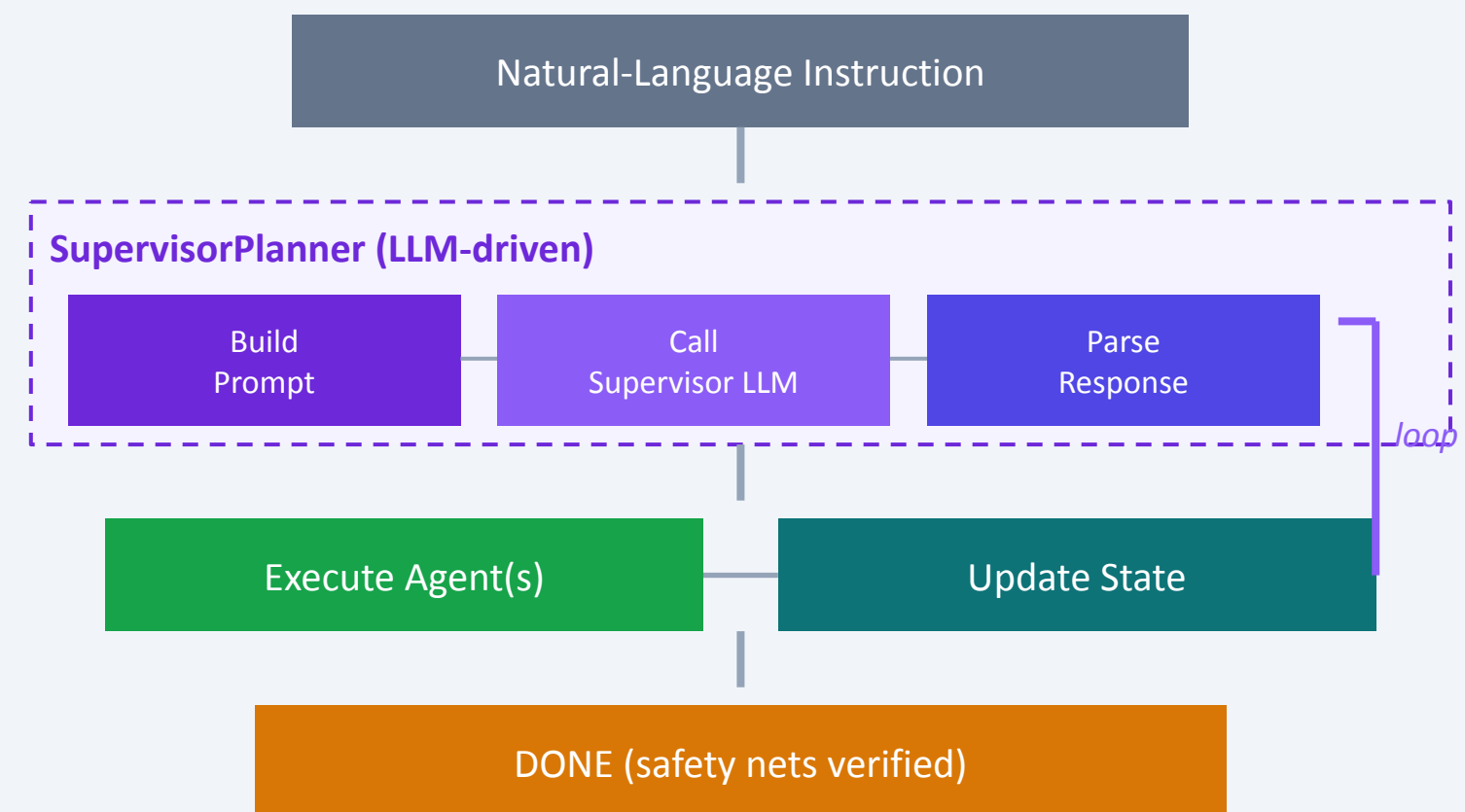
Workflow rules (intent-based), chain rules (structural), and workflow filter (output-based) bound the LLM's decisions — no steps skipped or added



5 Reusable Agents

5 agents (response, ranking, agreement, disagreement, synthesis) + 3 pure-computation aggregation actions

Iterative Planning Loop



Safety Nets: Before signaling DONE, the planner verifies

- (1) all workflow-required agents ran,
- (2) analysis stages have aggregates,
- (3) no out-of-scope agents were scheduled.

Pros & Cons

Supervisor LLM vs. fixed DAG and other orchestration approaches

ADVANTAGES

Natural-Language Control

Any instruction works — "just find what they agree on", "full deliberation", "quick consensus" — unlimited workflow flexibility

Dynamic Parallelism

Supervisor can schedule multiple agents on one line for concurrent execution — parallelism emerges from LLM decisions

3-Layer Safety Nets

Workflow rules, chain rules, and scope filters ensure the LLM can't skip required steps or over-schedule — bounded flexibility

Partial Workflows

Can run any subset of the 8 agents — no need for predefined compositions. The LLM adapts to user intent

Same Agent Codebase

Identical 8 agent implementations shared with fixed ADK mode — zero duplication between orchestration strategies

TRADE-OFFS

Extra LLM Cost

3–5 supervisor planning calls per deliberation add latency and API costs on top of the actual agent LLM calls

Non-Deterministic

Probabilistic LLM may choose different execution paths for the same instruction — not reproducible across runs

Parsing Fragility

Depends on the LLM producing exact "AGENTS: name" or "DONE" format — format deviations require fallback handling

Max Iterations Cap

Hard limit of 10 iterations prevents infinite loops but may truncate complex workflows if the LLM is inefficient

Keyword-Based Workflows

Workflow matching uses simple allMatch on keywords — ambiguous or novel instructions may not match any rule

Hierarchical Task Network Pattern

HTN Algorithm

Recursive top-down decomposition with greedy method selection

DECOMPOSE (task, world state)

Identify Task Type

PRIMITIVE TASK

- 1 Verify Preconditions**
Check that task preconditions hold in current world state
- 2 Execute & Add to Plan**
Append this primitive task to the output plan sequence
- 3 Update World State**
Apply task effects to advance the world state for subsequent tasks

COMPOUND TASK

- 1 Select Method**
Try in priority order; first whose precondition holds wins
- 2 Decompose into Subtasks**
Method produces ordered list of subtasks (compound/primitive)
- 3 Recurse on Each Subtask**
Call DECOMPOSE for each subtask; update state after each; merge sub-plans

↻ **Recursive — subtasks may themselves be compound**

Return Ordered Plan of Primitive Tasks

Greedy (no backtracking): first matching method commits the planner

Architecture

HTN (Hierarchical Task Network) Planning Pattern — LLM Council



Query Classifier

LLM-driven query classification into 6 categories (Factual, Opinion, Debate, etc.) to select the right pipeline



HTN Planner

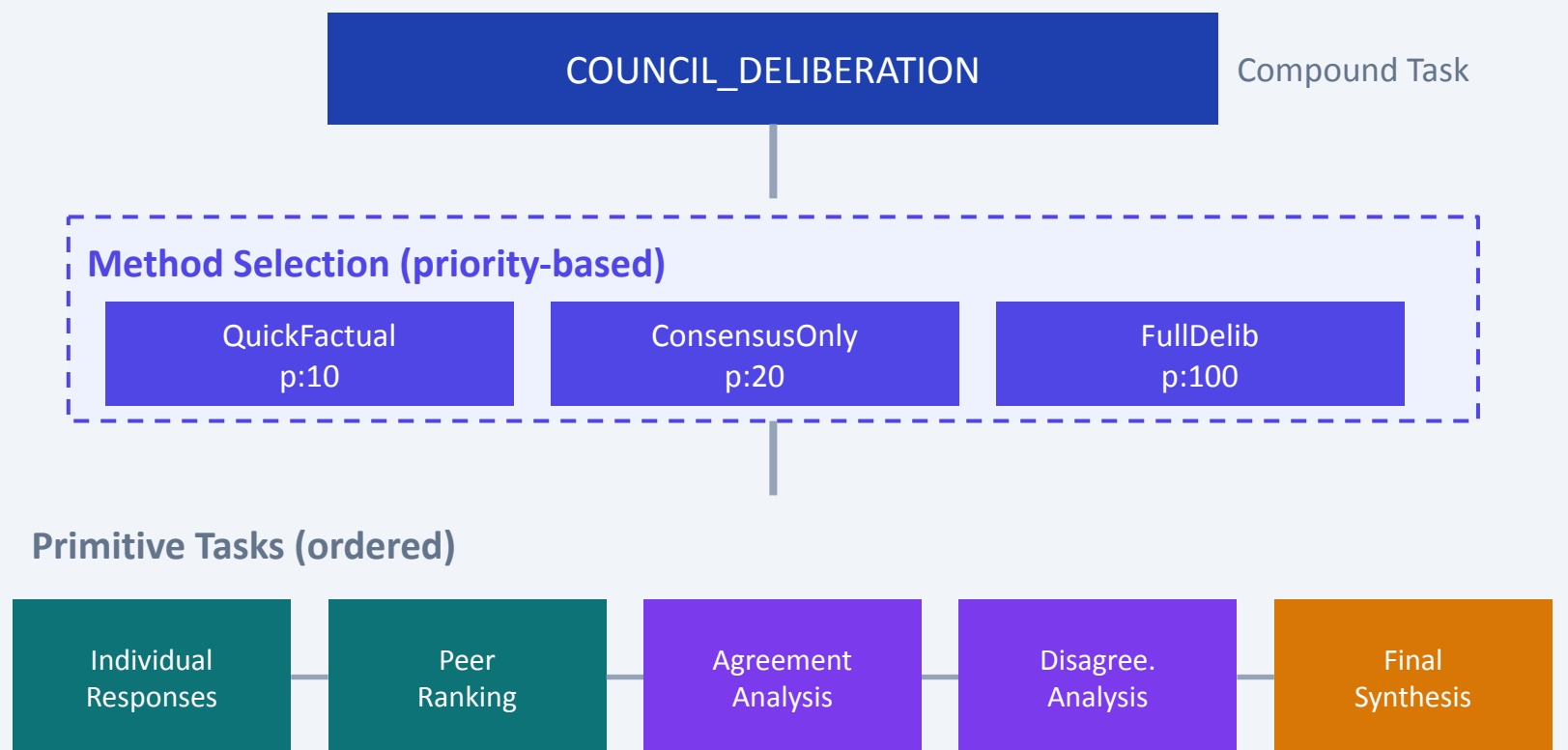
Recursive top-down decomposition of compound tasks into primitive tasks. Methods tried in priority order; first match wins



HTN Executor

Sequential dispatch of primitive tasks via reactive streams. Each stage completes before the next begins

HTN Decomposition



Pipeline varies by query type: Factual → 3 stages, Debate → 5 stages

HTN decomposes complex goals recursively into primitive tasks. The LLM classifier acts as the utility function that steers method selection.

Pros & Cons

HTN planning vs. fixed pipelines and other orchestration approaches

ADVANTAGES

Adaptive Pipelines

Not all queries need the same stages — HTN selects the right pipeline (3 or 5 stages) at runtime based on query type

Pure-Addition Extensibility

Adding a new strategy means adding one `@Component` bean. No existing code changes needed

Compile-Time Safety

Sealed interfaces + pattern matching guarantee exhaustive handling of all task types — the compiler catches gaps

Graceful Degradation

If classification fails, defaults to ANALYTICAL → full 5-stage pipeline runs as safe fallback. System always produces a plan

No Orchestration LLM Calls

Planning is pure computation over data structures — only the classifier costs one fast LLM call

TRADE-OFFS

No Backtracking

Greedy first-match selection — if a method produces a suboptimal plan, the planner cannot revise it mid-execution

Sequential Execution Only

Stages run one after another via `concatMap`. No parallel execution of independent stages (unlike Utility AI)

Classification Dependency

Pipeline quality depends entirely on the LLM classifier's accuracy — misclassification leads to wrong pipeline

Fixed Method Set

Methods are defined at compile time via Spring beans. No dynamic method generation or runtime composition

Confidence Not Used

Classifier returns confidence scores but methods don't use them — no threshold-based method switching

Utility AI Pattern

The OODA Execution Loop

Rapid decision-making in high-stakes, dynamic environments

OBSERVE

- **Filter eligible actions from world state**
- Check: preconditions met?
- Check: effects not yet available?
- Check: not already in-progress?

ORIENT

- **Score each eligible action**
- Readiness: how many actions unblocked?
- Progress: pipeline depth weighting
- Latency & Info Gain dimensions

DECIDE

- **Select actions scoring $\geq$ threshold (0.5)**
- Multiple actions can be selected
- If none above threshold, pick top scorer
- Guarantees forward progress

ACT

- **Execute selected actions in parallel**
- Mark effects as in-progress
- Flux.merge() for concurrent execution
- On complete: mark state available, recurse



Terminal: NIRVANA event when no eligible actions remain and nothing is in-progress

Architecture

Utility AI Planning Pattern — LLM Council Orchestration



World State

11 state variables tracked.
Actions check preconditions against available state.



Action System

9 concrete actions with preconditions, effects, cost (0.05–0.9) and value (0.5–1.0). Dependency graph encoded implicitly.



Scoring Engine

4 weighted scorers — Readiness, Progress, Latency, Info Gain — combined via CompositeScorer.

Evaluation Cycle

Query World State

Filter Eligible Actions

Score + Select

Execute in Parallel

Update World State

↻ Cycle repeats each decision tick until no eligible actions remain

Pros & Cons

Utility AI vs. alternative orchestration approaches

ADVANTAGES

- ✓ **Dynamic Parallelism**
Concurrent execution emerges naturally — no manual parallel wiring needed
- ✓ **Lightweight Decisions**
Scorer evaluations are $O(1)$ per action — no LLM calls for orchestration decisions
- ✓ **Implicit Dependencies**
Preconditions and effects encode the dependency graph without a central orchestrator
- ✓ **Adaptive Strategies**
3 scoring presets (balanced, latency-optimized, information-gain) swap with one config change
- ✓ **Full Observability**
Event stream traces every scoring decision — transparent, debuggable reasoning

TRADE-OFFS

- ✗ **Scorer Weight Tuning**
Weights require manual calibration — no auto-optimization of balanced vs. latency profiles
- ✗ **Fixed Action Set**
All 9 actions are predefined at compile time — no runtime action discovery or generation
- ✗ **Threshold Sensitivity**
The 0.5 parallel threshold affects concurrency; too low over-parallelizes, too high serializes
- ✗ **No Backtracking**
Actions are fire-and-forget — a failed action clears state but cannot undo downstream effects
- ✗ **Boolean State Only**
State is binary (available / not) — no partial completion, confidence levels, or rich state

9 sequential steps → 4 iterations with emergent parallelism

Goal-oriented action planning (GOAP)

The GOAP Algorithm

Goal-Oriented Action Planning — a graph search technique invented for game AI (F.E.A.R., 2005)

World State

Finite set of variables fully describe the current situation.
Each variable tracks one aspect of the env.
Planner reasons over this state to find valid action sequences.

Goals

Desired target values for one or more variables. Goal is satisfied when world state matches all required conditions. Multiple goal sets can coexist - planner selects the highest-priority unsatisfied goal to plan toward.

Actions (Preconditions + Effects)

Each action declares preconditions (world-state values required to execute) and effects (state variables it changes). Actions carry a numeric cost. Planner treats actions as edges in a state-space graph — only traversable when preconditions are met.

A* Search Process

1. Start Node = Current World State

2. Goal Node = State satisfying all goal conditions

3. Edges = Actions (if preconditions met)

4. Expand cheapest path first ($f = g + h$)

5. Admissible heuristic guarantees optimal plan

Result = Minimum-cost action sequence

Key: g = cost so far, h = heuristic estimate to goal. Admissible h never overestimates, so A* always finds the cheapest path.

Architecture

GOAP Planner — deterministic planning with auto-parallelism and runtime adaptive replanning



A* Search on 8 Actions

Finds cost-optimal action sequence from initial state {QUERY} to goal state. 5 LLM actions (cost 10 each) + 3 Java aggregation actions (cost 1 each). Admissible heuristic guarantees globally optimal plans.



10-Variable World State

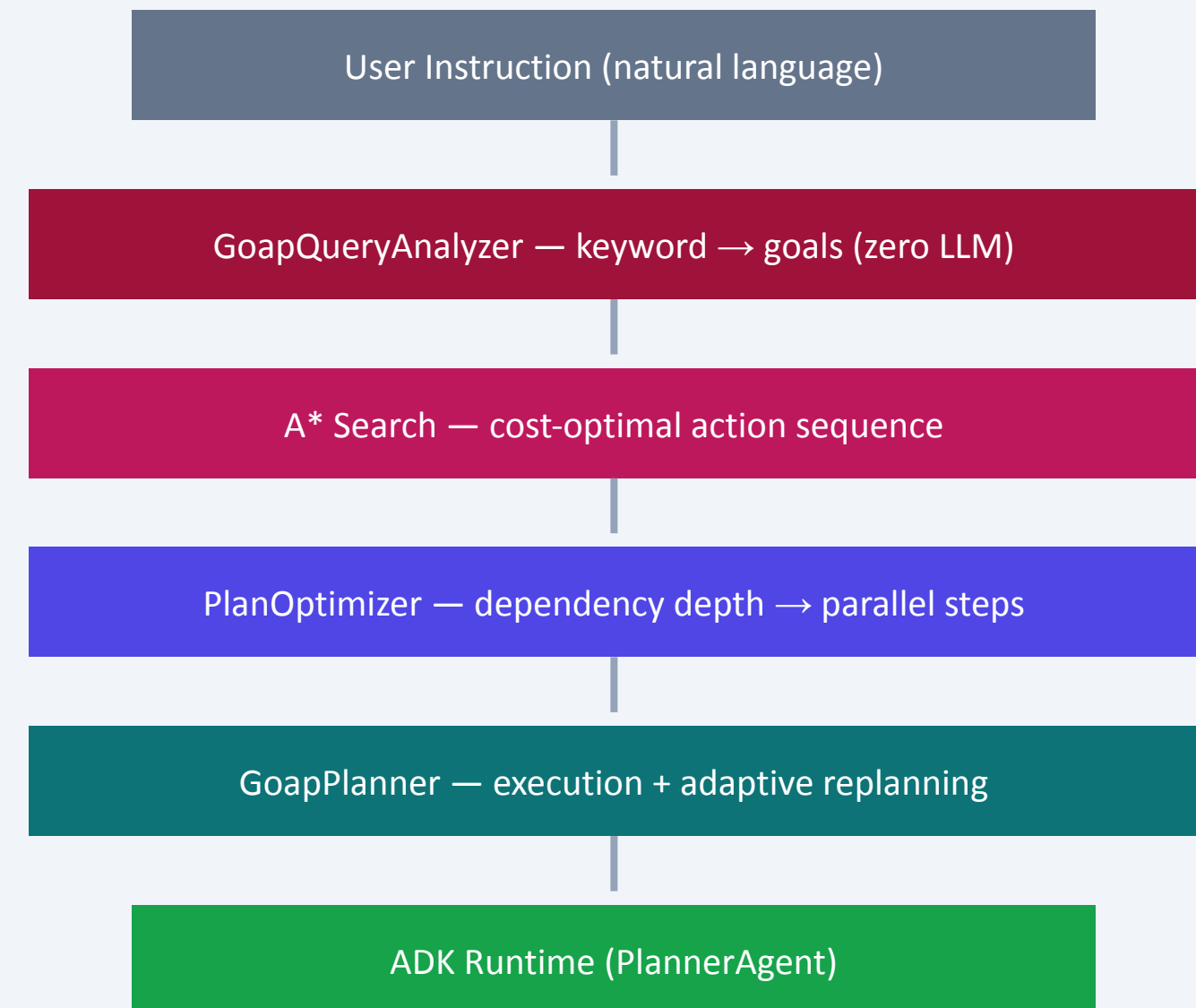
State variables track 10 data products (query, responses, rankings, analyses, aggregates). Actions have preconditions and effects on the variables



PlanOptimizer — Auto-Parallelism

Computes dependency depth per action — same depth run concurrently. 8 sequential A* actions collapse to 3 parallel steps. Adaptive replanning skip steps based on consensus mericst

Planning Pipeline



Pros & Cons

GOAP Planner vs. LLM Supervisor and fixed DAG orchestration approaches

ADVANTAGES

✓ Zero LLM Planning Cost

A* search runs entirely in Java — no LLM calls in the planner itself. Planning completes in microseconds vs. 100ms–2s per step for LLM supervisors.

✓ Fully Deterministic

Same goals always produce the same initial plan. Adaptive replanning is also deterministic (threshold comparisons, not LLM). Fully reproducible, auditable.

✓ Auto-Discovered Parallelism

PlanOptimizer computes dependency depths automatically — no hand-coded DAGs. 8 actions collapse to 3 parallel steps.

✓ Cost-Optimal Paths

A* with admissible heuristic guarantees globally cheapest plan.
LLM actions cost 10, Java aggregation costs 1 — A* minimizes total cost.

✓ Goal-Driven Flexibility

4 modes (Full, Standard, Quick, Ranking) selected by keyword matching — more flexible than fixed presets while staying deterministic.

TRADE-OFFS

✗ Keyword-Based Goal Selection

GoapQueryAnalyzer uses rule-based keyword matching — no NL understanding. "Help me analyze" won't trigger unless keywords match rules.

✗ Fixed Action Set

All 8 actions are statically declared in GoapActionRegistry. Adding a new action requires code changes — no runtime extensibility.

✗ Limited Runtime Adaptation

Adaptive replanning can skip steps based on consensus metrics (e.g., high Kendall's W skips disagreement aggregation), but cannot add new actions or reorder mid-run unlike LLM supervisors.

✗ A* Overhead Grows

State-space search is exponential in the number of actions.
Works well with 8 actions but could slow down with a larger action inventory.

✗ No Learning / Feedback Loop

Planner doesn't learn from past executions.
Same keywords always map to same goals — no improvement over time.

LLM Compiler Pattern

Architecture

LLM Compiler — one planning call emits the whole task DAG; a deterministic engine validates and runs it



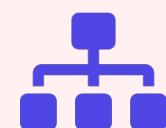
PlanCompiler — 1 LLM Call

One planning-LLM call emits the whole execution DAG as JSON. Prompt and validator share one source of truth.



PlanValidator Safety Net

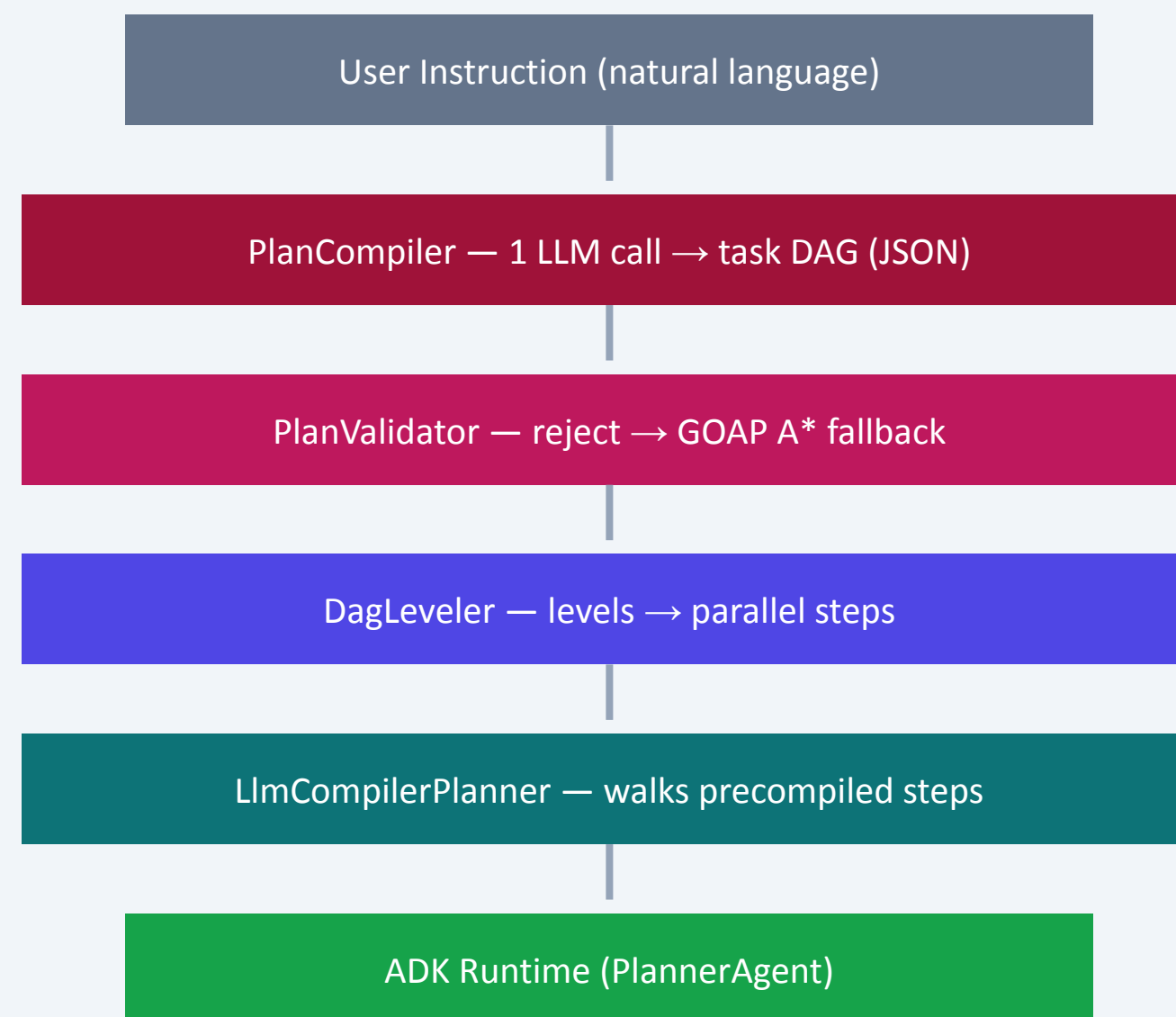
Checks that agents exist, IDs are unique, edges resolve, the graph is acyclic, preconditions close, and analysis → aggregate chains are present. Rejected plans fall back to GOAP A*.



DagLeveler — Parallelism

Levels the DAG by the edges the LLM authored — level = 1 + deepest dependency. Tasks sharing a level run concurrently as one ADK RunAgents step; 8 tasks → 3 steps

Compilation Pipeline



The LLM Compiler

Plan-and-Execute — one LLM authors the entire task graph up front, then a deterministic engine runs it

Plan Once, Not Per Step

The planning LLM is prompted a single time and returns the complete plan as JSON: {label, tasks[{id, agent, dependsOn}]}. A supervisor is re-prompted after every step; here planning cost is $O(1)$, not $O(\text{steps})$.

Explicit Task DAG

Each task declares the ids of the tasks it depends on. Parallelism comes from the edges the model authored, not from static preconditions and effects — the LLM chooses the graph shape, the engine only honours it.

Validate, Then Execute As-Is

A hallucinated plan never reaches execution. The static action registry verifies the DAG rather than building it; if verification fails, a deterministic GOAP A* plan takes over so a run always has something runnable.

Execution Flow

1. One planning call → JSON task DAG

2. Validate: agents, ids, cycles, closure, chains

3. Level the DAG → ordered parallel steps

4. Emit plan preview, then execute in ADK

5. Between runs: consensus-driven replan

Result = LLM-authored plan, deterministic run

Key: Replanning happens between whole runs, not per step

Pros & Cons

LLM Compiler vs. GOAP planning and step-by-step LLM supervision

ADVANTAGES

✓ One Planning Call

The full DAG arrives in a single LLM round trip — planning cost stays flat as the plan grows, versus one call per step for a supervisor.

✓ Semantic Intent Understanding

The model reads intent in natural language, so “help me weigh the options” maps to a sensible plan without keyword rules or fixed modes.

✓ LLM-Authored Parallelism

Concurrency comes from the explicit dependsOn edges the model wrote — it can group work that static precondition analysis would serialize.

✓ Validated Before Execution

PlanValidator checks agents, unique ids, cycles, precondition closure and mandatory aggregate chains — bad plans are rejected, never run.

✓ Deterministic Fallback

Malformed JSON or a rejected plan falls back to GOAP A* over the same registry, so a deliberation always has a runnable plan.

TRADE-OFFS

✗ Non-Deterministic Plans

The same instruction can compile to different DAGs across runs. Reproducibility depends on the model, not on a deterministic search.

✗ Planning Latency and Cost

One LLM call sits on the critical path before any agent runs — microseconds of A* search become hundreds of milliseconds to seconds.

✗ No Cost-Optimality Guarantee

Nothing minimizes total action cost. The model can pad the plan or pick a more expensive route — the validator enforces validity, not optimality.

✗ Coarse Replanning

Replanning fires at most once, between whole runs, on a fixed consensus rule — the plan cannot be revised mid-run as steps reveal new information.

✗ Prompt-Coupled Correctness

Plan quality tracks prompt and catalog quality. Adding an agent means prompt, registry and validator all have to stay in sync.

Why? ... When...?

Why These Patterns

Same agents, same CouncilService — six different answers to “which agents, in what order, with what parallelism?”

Workflow — Composition Is a Fact

When the pipeline never changes, encoding it at compile time is the honest answer. Sequential / Parallel agent trees make the DAG readable in the source, cost nothing to plan, and are trivially reviewable.

Supervisor — Intent in Plain Language

Free-form instructions are the cheapest interface for a human. An LLM reads the request and picks the next agents step by step, so nothing has to be enumerated in advance.

Utility AI — No Plan Survives the Run

Some decisions only make sense with live data. The OODA loop re-scores every eligible action each iteration on readiness, progress, latency and information gain, so order emerges instead of being fixed.

HTN — Expertise as Recipes

Domain knowledge is often already shaped like a recipe. Each HtnMethod is a hand-authored decomposition with a precondition and a priority — a new strategy is one Spring bean, no registry to edit.

GOAP — The Cheapest Correct Plan

When cost matters and goals are state, search beats prompting. A* over preconditions and effects with an admissible heuristic returns the optimal plan at zero LLM cost, and parallelism falls out of the data.

LLM Compiler — Understanding, Once

You want the model's grasp of intent without paying for it at every step. One call authors the whole DAG; PlanValidator and a GOAP fallback make an LLM-authored plan safe to execute as written.

Deterministic, zero planning cost · Workflow → HTN → GOAP | Utility AI | LLM Compiler → Supervisor · Adaptive, LLM-driven



When to Use

Reach for the simplest planner that still explains the variation in your workload

Workflow — Fixed, Known Pipelines

Use when the shape of the work never varies and a preset (full / quick / rank) is enough. Avoid the moment the right agent set depends on the request.

Supervisor — Open-Ended Exploration

Use for prototyping and free-form instructions where flexibility beats everything else. Avoid on production paths that care about latency, token cost, or getting the same answer twice.

Utility AI — Conditions Over Plans

Use when latency budgets, partial results or adaptive parallelism should steer execution and you can tune the weights. Avoid when you owe someone an auditable plan up front.

HTN — A Few Known Query Classes

Use when you can name the strategies — factual, informational, opinion, debate — and experts can write the recipes. Avoid when the strategy space is too large or too fuzzy to enumerate.

GOAP — Goals as State, Cost as Budget

Use when intent maps to target state variables and you need reproducible, auditable, cost-optimal plans at zero planning cost. Avoid when the instruction needs real language understanding.

LLM Compiler — Free-Form, Bounded Cost

Use when instructions are open-ended but you still want one flat planning cost, a plan preview, and validation with a deterministic fallback. Avoid under strict reproducibility rules.

Default to Workflow. Add GOAP when the goals vary, LLM Compiler when the language does, Utility AI when the runtime does.



Single axis representation for planning algorithms

From strict compliance all the way to bounded flexibility

◀ Strict Compliance

Hybrid approach

Bounded Flexibility ▶

Workflows

Deterministic Directed
Acyclic Graphs (DAGs)

0 planning LLM calls

HTN

Hierarchical Task Network
planning

1 call · classify only

GOAP

Goal-Oriented Action
Planning via A* graph search

0 calls · keyword rules

Utility AI

Scored decision-making
across weighted options

0 calls · scoring fn

LLM Compiler

LLM-authored task DAG,
deterministically verified

1 call · authors the DAG

SupervisorLLM

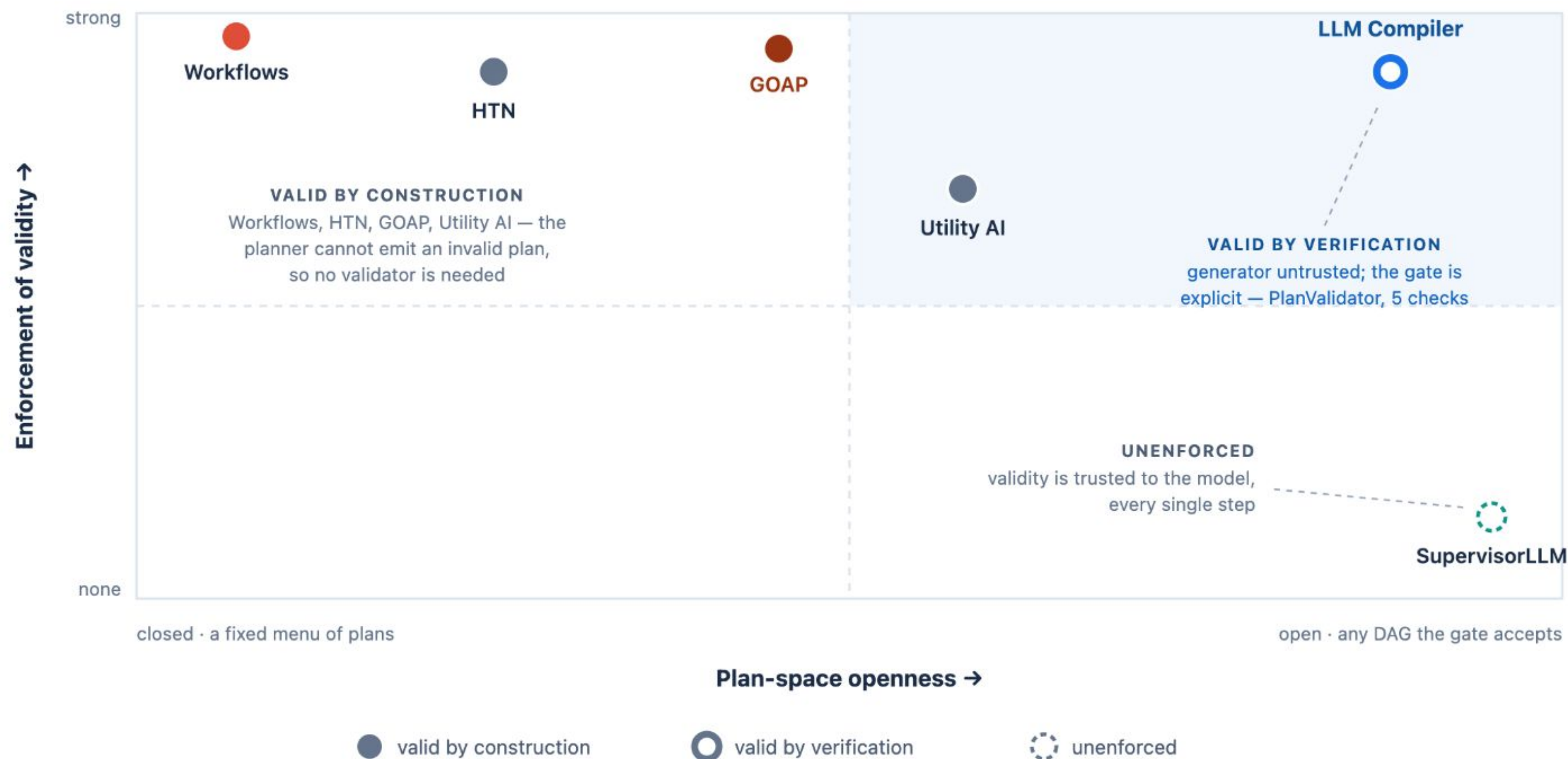
LLM-orchestrated
delegation to sub-agents

1 call per step



We need a multi-axis representation

From strict compliance all the way to bounded flexibility



Models will keep changing. New patterns will keep emerging. The best thing we can do is stay “architectural” and apply the right pattern for the problem in front of us - not the one that’s trending

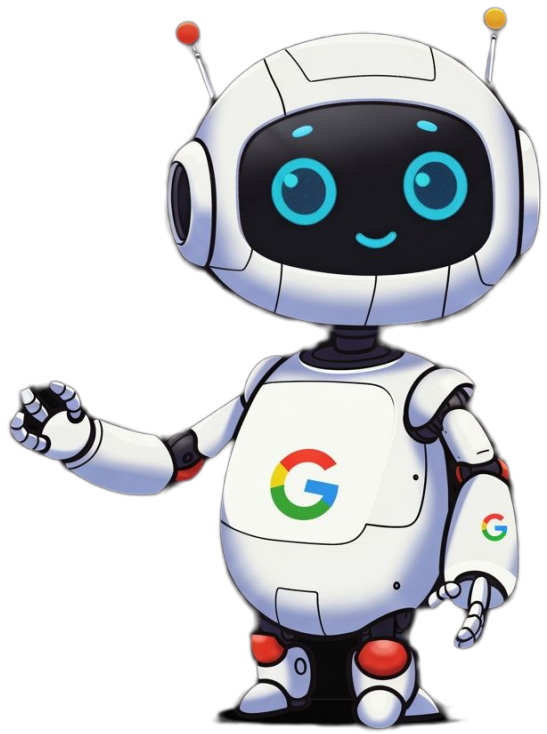
Remember:

The best pattern might be the one you build next



Thank you

@ddobrin



Google Cloud